

PyXtabond2 User Manual

Dynamic Panel GMM Estimation in Python

Frederic Kahindo

Version 0.0.4 — September 6, 2026

Abstract

`pyxtabond2` is a Python package for estimating dynamic panel data models by the Generalized Method of Moments (GMM), built to replicate the matrix algebra, options, and diagnostics of Stata's `xtabond2` [Roodman, 2009a]. It covers Difference GMM [Arellano and Bond, 1991] and System GMM [Arellano and Bover, 1995, Blundell and Bond, 1998], one- and two-step estimation with the Windmeijer [2005] finite-sample correction, Forward Orthogonal Deviations, instrument collapsing, and a full battery of specification diagnostics. It also adds an original extension not present in Stata's `xtabond2`: *IFE-GMM*, an iterative PCA-based estimator for dynamic panels contaminated by unobserved interactive fixed effects [Bai, 2009], with an optional split-panel jackknife bias correction [Dhaene and Jochmans, 2015]. This manual documents the package's constructor options, diagnostics, export paths, and known limitations as of v0.4.1. The underlying econometric theory, the line-by-line validation against Stata, and the validity analysis of IFE-GMM are the subject of a companion Methodological Note; this manual is deliberately practical.

Contents

1	Introduction	2
1.1	Key features	2
1.2	Package status	2
2	Installation	3
3	Quick start	3
4	Preparing panel data: <code>PanelData</code>	4
5	The <code>PyXtabond2</code> estimator	4
5.1	Overview	4
5.2	Core data parameters	5
5.3	Instrument specification	5
5.4	Model type and estimation method	6
5.5	Weighting-matrix and test controls	7
5.6	IFE-GMM parameters (overview)	7
5.7	Full parameter reference	7
6	Python ↔ Stata <code>xtabond2</code> option mapping	8
7	Estimation results: <code>PyXtabond2Results</code>	9
7.1	<code>.summary()</code>	9
7.2	Interpreting the diagnostics	10

8	Exporting results	10
8.1	Comparing multiple models: <code>GMMStargazer</code>	10
9	IFE-GMM: the interactive-fixed-effects extension	11
9.1	Motivation	11
9.2	Parameters	11
9.3	Convergence and bias caveats	12
10	Known limitations	12
11	Performance notes	13
12	Worked examples	13
12.1	Standard GMM: <code>examples.py</code>	13
12.2	IFE-GMM: <code>ife_exemples.py</code>	15
12.3	Bundled datasets	16

1 Introduction

`pyxtabond2` targets applied econometricians who want a Python-native dynamic panel GMM workflow without giving up the option surface and diagnostics that Stata’s `xtabond2` [Roodman, 2009a] made a de facto standard. This manual serves two audiences:

- Stata users porting a `xtabond2` specification to Python, who want a direct option-by-option mapping (Section 6).
- Researchers working with panels where unobserved *common factors* (interactive fixed effects) are a concern, who want an estimator that purges them before GMM estimation (Section 9).

1.1 Key features

- Difference GMM (Arellano-Bond) and System GMM (Blundell-Bond / Arellano-Bover), one-step and two-step, with all four robust/non-robust combinations Stata supports.
- Windmeijer [2005] finite-sample correction for two-step robust standard errors.
- Forward Orthogonal Deviations [Arellano and Bover, 1995], ideal for unbalanced panels with gaps.
- Full support for **unbalanced panels** – missing (`id`, `time`) rows, not just missing values inside an existing row – throughout estimation, in both the standard GMM engine and IFE-GMM (Section 4).
- Instrument collapsing to control instrument proliferation [Roodman, 2009b].
- Generalized `h()`, `artests()`, `arlevels`, and single-variable `cluster()` options, matching `xtabond2.mata`.
- Per-variable-group instrument suboptions (`lag()`, `collapse`, `eq()`) via `GMMStyle/IVStyle`.
- **IFE-GMM**: an original iterative PCA-defactoring GMM estimator for panels with interactive fixed effects [Bai, 2009], with automatic factor-count selection [Bai and Ng, 2002, Ahn and Horenstein, 2013] and a split-panel jackknife bias correction [Dhaene and Jochmans, 2015].
- Publication-ready export to L^AT_EX and Microsoft Word, including multi-model comparison tables.

1.2 Package status

`pyxtabond2` is at version 0.4.1 (pre-1.0): the API may still change between minor versions as features are added. Supported Python versions are 3.9–3.12. Every numeric feature de-

scribed in this manual has been cross-checked against a fixed regression-fixture suite and, where a Stata equivalent exists, against a real Stata run of `xtabond2` on the same data; see the companion Methodological Note for the full validation record. (The underlying `tests/` and `stata_validation/` material lives in the project's development repository and is not necessarily included in every source distribution.)

2 Installation

```
pip install pyxtabond2
```

Word export needs the optional `export` extra (the bundled example datasets are plain CSV and need no extra dependency):

```
pip install "pyxtabond2[export]"
```

Development install from source:

```
git clone https://github.com/Kahindo048/pyxtabond2.git
cd pyxtabond2
pip install -e ".[dev,export]"
```

Dependency	Minimum version	Role
numpy	1.21	Linear algebra core
pandas	1.3	Panel data handling
scipy	1.7	SVD, distributions
statsmodels	0.13	Summary table formatting
matplotlib	3.5	Factor-selection plots
numba	0.56	JIT-compiled panel transforms
python-docx (extra: export)	0.8	Word export
pytest (extra: dev)	7.0	Test suite

3 Quick start

`pyxtabond2` bundles example datasets so you can experiment immediately.

```
from pyxtabond2 import PanelData, PyXtabond2, load_dataset

# 1. Load the data
df = load_dataset('df_panel.csv')

# 2. Prepare the panel and precompute the lagged dependent variable
# (Python has no live L./D. operators inside the estimation call --
# lags used as *regressors* must be precomputed and merged in).
panel = PanelData(df, id_col='Country', time_col='Year')
panel.data['L1_Growth'] = panel.get_lag('Growth', 1)
df_ready = panel.data.reset_index()

# 3. Specify and estimate the model
model = PyXtabond2(
    df_ready,
    id_col='Country', time_col='Year', dep_var='Growth',
    x_vars=['L1_Growth', 'Capital', 'Labor', 'Wage', 'Investment', 'Ide'],
    gmm_vars=['Growth', 'Capital'], # Arellano-Bond (GMM-style) instruments
```

```

    iv_vars=['Ide'],                      # standard IV instruments
    model_type='system', twostep=True, robust=True, small=True,
)
result = model.fit()
result.summary()

# 4. Export for publication
result.to_latex("gmm_results.tex", full_output=False)
result.to_word("gmm_results.docx", full_output=False)

```

See Section 12 for the full four-specification workflow shipped as `examples.py` (in `examples/`).

4 Preparing panel data: `PanelData`

`PanelData` is the equivalent of Stata's `tsset`: it sorts the data by (`id_col`, `time_col`), builds a `MultiIndex`, drops rows with a missing identifier or time period (with a warning), and pre-computes per-group row slices used throughout estimation.

```

panel = PanelData(df, id_col='Country', time_col='Year')
panel.data          # cleaned, sorted, MultiIndexed DataFrame

```

Three within-group temporal operators are provided, all NaN-aware (a group with gaps does not contaminate a neighbouring group, and missing values inside a group are simply skipped rather than propagated):

Method	Description
<code>get_lag(var, lags=1)</code>	Stata's <code>L.var</code> / <code>L2.var</code> .
<code>get_first_difference(var)</code>	Stata's <code>D.var</code> .
<code>get_fod(var)</code>	Arellano-Bover (1995) forward orthogonal deviations: subtracts the average of all <i>available future</i> observations (rather than just the previous one), minimizing data loss in unbalanced panels with gaps.

Caution. Python has no live `L./D.` operator evaluated at estimation time. Lags of the dependent variable that you want to use as a *regressor* (e.g. `L1.Growth` in `x_vars`) must be precomputed with `get_lag` and merged into the dataframe you pass to `PyXtabond2`, as in the Quick Start above. This is distinct from the GMM-style instruments generated automatically for `gmm_vars/gmm` (their own internal lag structure is built by `SystemGMMBuilder` and needs no manual precomputation).

Note. Panels with missing (`id`, `time`) rows (an individual skipping a period entirely, joining late, or leaving early) are fully supported throughout estimation – by the temporal operators above, the GMM engine, and IFE-GMM alike – mirroring Stata `xtabond2`'s own internal dense-grid handling of unbalanced data. No option is needed: gaps are detected automatically from `id_col/time_col`.

5 The `PyXtabond2` estimator

5.1 Overview

`PyXtabond2` is the single entry point for every model the package supports. Construct it with your data and specification, then call `.fit()`:

```

model = PyXtabond2(df, id_col, time_col, dep_var, x_vars, ...)
result = model.fit() # -> PyXtabond2Results (Section 7)

```

`.fit()` routes automatically: if `r=0` (the default), it runs the standard GMM engine directly; if `r` is a positive integer or `'auto'`, it runs the IFE-GMM iterative loop instead (Section 9). Every other option below applies identically in both cases.

5.2 Core data parameters

`df`, `id_col`, `time_col`, `dep_var`, and `x_vars` specify, respectively: the panel (long format, one row per unit-period), the cross-sectional identifier column, the time column, the dependent variable, and the list of strictly exogenous regressors (which may include precomputed lags, as above). These are the only five required, positional arguments.

5.3 Instrument specification

Two variable roles map onto Stata's two instrument families:

- **GMM-style instruments** (`gmm_vars` / `gmm`): Arellano-Bond-style instruments built from a variable's own lags (endogenous or predetermined variables).
- **Standard IV instruments** (`iv_vars` / `iv`): a variable used as its own instrument (strictly exogenous variables).

Each family has a flat *shortcut* form and an advanced *per-group* form; the advanced form takes precedence when given.

Shortcut form. `gmm_vars` (list of column names) plus a single `lag_limits_diff` (default (2, None), meaning "lags 2 to the maximum available") and a single `collapse` flag (default False) apply uniformly to every GMM-style variable. `iv_vars` is just a list of column names.

Advanced per-group form. Pass `gmm=[GMMStyle(...), ...]` and/or `iv=[IVStyle(...), ...]` to control lag windows, collapsing, and equation targeting *per variable group*, mirroring Stata's repeated `gmm()/iv()` option syntax:

```
from pyxtabond2 import GMMStyle, IVStyle

model = PyXtabond2(
    df, id_col='country_id', time_col='Year', dep_var='Growth',
    x_vars=['L.Growth', 'Capital', 'Labor', 'Wage', 'Investment', 'Ide'],
    gmm=[
        GMMStyle('Growth', lag=(2, 4), collapse=True),
        GMMStyle('Capital', lag=(2, None)),
    ],
    iv=[IVStyle('Ide')],
    model_type='system',
)
```

This reproduces (exactly, cross-checked against a real Stata run) Stata's `gmm(Growth, lag(2 4) collapse) gmm(Capital, lag(2 .)) iv(Ide)`.

GMMStyle field	Default	Meaning
variables	required	Variable(s) in this group (str or list).
lag	(2, None)	(min_lag, max_lag); None means "as many as available" (Stata's .).
collapse	False	Collapse into one instrument column per lag distance instead of one per (time period × lag).
equation	"both"	"both", "diff", or "level" – which equation(s) this group instruments. Only meaningful for <code>model_type='system'</code> (see caution below).

IVStyle field	Default	Meaning
variables	required	Variable(s) in this group.
equation	"both"	Same as <code>GMMStyle.equation</code> .

Caution. Under `model_type='difference'`, `GMMStyle.equation` is always silently ignored: Difference GMM only ever has a differenced equation, so there is nothing to select between. `IVStyle.equation` follows the same rule for "diff" and "both", but `IVStyle(..., equation="level")` raises a `ValueError` instead of being ignored, since it would leave the instrument assigned to no equation at all. Use "diff" (or the default "both") for any `IVStyle` group under Difference GMM. `equation` only meaningfully selects between equations for `model_type='system'`.

Note. Stata's further `passthru`, `mz`, and `split` suboptions of `gmm()/iv()` are **not implemented** (see Section 10).

5.4 Model type and estimation method

Parameter	Default	Meaning
model_type	'difference'	'difference': Arellano-Bond [Arellano and Bond, 1991]. 'system': Blundell-Bond / Arellano-Bover [Arellano and Bover, 1995, Blundell and Bond, 1998], adding the levels equation instrumented by lagged differences.
twostep	False	Two-step efficient GMM instead of one-step.
robust	False	Robust (sandwich) standard errors; combined with <code>twostep</code> , applies the Windmeijer [2005] finite-sample correction.
orthogonal	False	Forward Orthogonal Deviations instead of first differences.
small	False	Small-sample degrees-of-freedom corrections: t/F statistics instead of z/χ^2 .

`twostep` and `robust` combine into four distinct estimators, exactly as in Stata:

twostep	robust	Estimator	Variance source
False	False	One-step, non-robust (homoscedastic)	Analytic V_1 scaled by an estimated error variance $\hat{\sigma}^2$.
False	True	One-step, robust (sandwich)	Robust sandwich $V_{1,\text{rob}}$; the Hansen test is additionally obtained via a silent two-step-style refit ("Roodman's trick"), without altering the reported one-step $\hat{\beta}/\text{se}$.
True	False	Two-step, non-robust (rare)	$\sqrt{\text{diag}(V_2)}$, no Windmeijer correction – Stata allows this combination but it is seldom used in practice.

twostep	robust	Estimator	Variance source
True	True	Two-step, robust = Windmeijer [2005]	Windmeijer-corrected $V_{2,\text{robust}}$ – the estimator most commonly reported in applied work.

Note. System GMM automatically adds a constant. When `model_type='system'`, a constant term (`_cons`) is automatically appended to the regressors and instrumented in the levels equation – you do not need to (and should not) add it to `x.vars` yourself. Difference GMM never has a constant term, since it cancels identically under differencing (matching Stata exactly). You will see `_cons` appear in the coefficient table of any System GMM result.

5.5 Weighting-matrix and test controls

Parameter	Default	Meaning
<code>h</code>	3	One-step weighting-matrix structure (Stata's <code>h()</code>). <code>h=3</code> (default) models the known cross-covariance between the differenced/orthogonal-deviations equation and the levels equation. <code>h=2</code> assumes that cross-covariance is zero. <code>h=1</code> assumes no serial-correlation structure at all (identity weighting). <code>h=1/h=2</code> are rarely needed in practice; <code>h=2</code> and <code>h=3</code> coincide for pure Difference GMM.
<code>artests</code>	2	Highest-order Arellano-Bond $AR(\ell)$ serial-correlation test reported (default: $AR(1)$ and $AR(2)$). Raise this to probe higher-order serial correlation directly.
<code>arlevels</code>	False	If True , the $AR(\ell)$ tests target the <i>levels</i> -equation residuals instead of the differenced/orthogonal-deviations residuals. Only valid for <code>model_type='system'</code> (raises <code>ValueError</code> otherwise).
<code>cluster</code>	None	Column name to cluster the robust/two-step variance matrix by, instead of the default panel id. Single variable only – see Section 10 for the multi-way limitation and a numerical caveat when instruments outnumber clusters.

5.6 IFE-GMM parameters (overview)

`r`, `r_max`, `ife_max_iter`, `ife_tol`, and `bias_correction` control the original IFE-GMM extension and are detailed in full in Section 9; `r=0` (the default) simply disables IFE-GMM and every model in this manual up to that point uses standard GMM.

5.7 Full parameter reference

Parameter	Type	Default	Description
<code>df</code>	DataFrame	required	Panel dataset (long format).
<code>id_col</code>	str	required	Cross-sectional identifier column.
<code>time_col</code>	str	required	Time-period column.
<code>dep_var</code>	str	required	Dependent variable.
<code>x.vars</code>	list[str]	required	Strictly exogenous regressors.

Parameter	Type	Default	Description
<code>gmm_vars</code>	<code>list[str]</code>	<code>None</code>	Shortcut GMM-style instrument variables (single group).
<code>iv_vars</code>	<code>list[str]</code>	<code>None</code>	Shortcut standard-instrument variables (single group).
<code>model_type</code>	<code>str</code>	<code>'difference'</code>	<code>'difference'</code> or <code>'system'</code> .
<code>twostep</code>	<code>bool</code>	<code>False</code>	Two-step instead of one-step GMM.
<code>robust</code>	<code>bool</code>	<code>False</code>	Robust/Windmeijer standard errors.
<code>lag_limits_diff</code>	<code>tuple</code>	<code>(2, None)</code>	Shortcut lag window for <code>gmm_vars</code> .
<code>collapse</code>	<code>bool</code>	<code>False</code>	Shortcut collapse flag for <code>gmm_vars</code> .
<code>gmm</code>	<code>list[GMMStyle]</code>	<code>None</code>	Advanced per-group GMM-style instruments; overrides the shortcuts above.
<code>iv</code>	<code>list[IVStyle]</code>	<code>None</code>	Advanced per-group standard instruments; overrides <code>iv_vars</code> .
<code>orthogonal</code>	<code>bool</code>	<code>False</code>	Forward Orthogonal Deviations instead of first differences.
<code>small</code>	<code>bool</code>	<code>False</code>	Small-sample dof corrections (t/F vs. z/χ^2).
<code>r</code>	<code>int</code> or <code>'auto'</code>	<code>0</code>	Number of IFE-GMM factors; <code>0</code> = plain GMM.
<code>r_max</code>	<code>int</code>	<code>5</code>	Max factors tested when <code>r='auto'</code> .
<code>ife_max_iter</code>	<code>int</code>	<code>30</code>	Max IFE-GMM loop iterations.
<code>ife_tol</code>	<code>float</code>	<code>1e-5</code>	IFE-GMM convergence tolerance.
<code>bias_correction</code>	<code>bool</code>	<code>False</code>	Split-panel jackknife bias correction on top of IFE-GMM.
<code>h</code>	<code>{1,2,3}</code>	<code>3</code>	One-step weighting-matrix structure.
<code>artests</code>	<code>int</code>	<code>2</code>	Highest $AR(\ell)$ lag tested.
<code>arlevels</code>	<code>bool</code>	<code>False</code>	$AR(\ell)$ tests on levels residuals (system only).
<code>cluster</code>	<code>str</code>	<code>None</code>	Single clustering variable for robust/two-step inference.

6 Python ↔ Stata xtabond2 option mapping

Caution. Default model type differs. `pyxtabond2`'s own default is `model_type='difference'`. Stata's `xtabond2` default is *System* GMM (the level equation is included unless `noleveleq` is given). A direct Stata-to-Python port therefore needs `model_type='system'` explicitly unless the Stata call used `noleveleq`.

pyxtabond2	Stata xtabond2	Notes
<code>model_type='system'</code>	default (no <code>noleveleq</code>)	See caution above: opposite defaults.
<code>model_type='difference'</code>	<code>noleveleq</code>	<code>pyxtabond2</code> 's own default.
<code>gmm_vars</code> <code>gmm=[GMMStyle(...)]</code>	/ <code>gmm(varlist,</code> <code>lag(a b) collapse</code> <code>eq(diff level both))</code>	One <code>GMMStyle</code> group = one <code>gmm()</code> call; <code>lag=(a,b) ↔ lag(a b)</code> (<code>None ↔ Stata's .</code>).
<code>iv_vars</code> <code>iv=[IVStyle(...)]</code>	/ <code>iv(varlist,</code> <code>eq(diff level both))</code>	Analogous to <code>gmm()</code> .

pyxtabond2	Stata xtabond2	Notes
twostep	twostep	Direct.
robust	robust	Direct, including the one-step "Roodman's trick" Hansen refit.
orthogonal	orthogonal	Direct (FOD vs. first differences).
small	small	Direct.
h=1/2/3	h(1)/h(2)/h(3)	Direct, same default (3).
artests=#	artests(#)	Direct, same default (2).
arlevels	arlevels	Direct, System-only in both.
cluster=var	cluster(varname)	pyxtabond2: single variable only, <i>not</i> Stata's multi-way cluster(var1 var2).
r, r_max, ife_max_iter, ife_tol, bias_correction	<i>no equivalent</i>	Original pyxtabond2 extension (IFE-GMM, Section 9); not part of xtabond2. The closest Stata-world tools are the community-contributed xtivdfreg (two-stage de-factored IV, Kripfganz and Sarafidis, 2021 , Norkutė et al., 2021) or xtdcce2 (Common Correlated Effects, Ditzen, 2018 , Pesaran, 2006) – different estimators, not drop-in equivalents.
<i>not available</i>	pweight/aweight/fweight	Not implemented (Section 10).
<i>not available</i>	multi-way cluster(var1 var2)	Not implemented (Section 10).
<i>not available</i>	gmm()/iv()'s passthru, mz, split suboptions	Not implemented.

7 Estimation results: PyXtabond2Results

.fit() returns a PyXtabond2Results instance.

Attribute	Description
.beta, .se	Coefficient and standard-error column vectors, aligned with .x_names.
.t_stats, .p_values	t/z statistics and two-sided p -values.
.ci_lower, .ci_upper	95% confidence interval bounds.
.x_names	Regressor names, in output order (includes _cons for System GMM).
.diag	Dictionary of diagnostic tests (see below); advanced use.
.engine	The underlying GMMEngine; advanced/internal use (raw matrices, group structure).

7.1 .summary()

`result.summary()` prints a Stata-style report with four blocks:

1. **Header:** dependent variable, model/method, date/time, observation and group counts, obs. per group, instrument count, covariance type, the Wald/ F statistic and its p -value, and the active specification (GMM/IV variable lists, lag limits, collapse, FOD, small-sample correction).
2. **Coefficient table:** coefficient, standard error, t/z statistic, p -value, and 95% CI for each regressor.
3. **Diagnostic Tests:** $\text{AR}(\ell)$ for $\ell = 1, \dots$, `artests`, the Sargan test, and (when available) the Hansen test.
4. **Difference-in-{Sargan,Hansen} Tests:** one row pair for the System GMM "GMM instruments for levels" block (when applicable), plus one further row pair covering all `iv()/iv_vars` instruments jointly. Every standard-instrument variable, across every `IVStyle` group, is tested together as a single combined subset – not group by group.

7.2 Interpreting the diagnostics

AR(1)/AR(2) Arellano-Bond serial-correlation tests on the transformed-equation residuals (H_0 : no serial correlation at that order). Differencing (or FOD) mechanically induces first-order correlation in well-specified errors, so **rejecting AR(1) is expected and not a problem**. Failing to reject AR(2) or higher, however, is required for validity of lag-2-and-beyond instruments; a rejected AR(2) signals that the original error term was itself serially correlated, invalidating those instruments. Raise `artests` to inspect AR(3), AR(4), ... directly.

Sargan vs. Hansen Both are overidentification tests of instrument validity (H_0 : instruments are valid). The Sargan test is always computed but is only valid under homoskedasticity; the Hansen test is consistent under the active robust/two-step weighting matrix (available whenever `twostep` or `robust` is set, including the one-step-robust "Roodman's trick" case) and should be preferred when present. Both tests **lose power as instruments proliferate** [Roodman, 2009b] – a suspiciously high Hansen p -value (e.g. > 0.99) with many instruments is itself a warning sign rather than reassurance; consider `collapse=True`.

Difference-in-Sargan/Hansen Tests the exogeneity of a specific instrument *subset* by comparing the overidentification statistic with and without it. A large difference p -value supports treating that subset as valid on its own.

Wald / F test Joint significance of all regressors (`_cons` excluded from the tested set and from its degrees of freedom when present). Reported as an F statistic under `small=True`, as χ^2 otherwise.

8 Exporting results

```
result.to_latex("table.tex", full_output=False) # booktabs academic table
result.to_latex("appendix.tex", full_output=True) # verbatim console capture
result.to_word("table.docx", full_output=False) # requires pyxtabond2[export]
```

`full_output=False` (default) produces a clean coefficients table plus a diagnostics table, formatted with booktabs rules or native Word table styles, suitable for direct inclusion in a paper. `full_output=True` instead captures the exact `.summary()` console output verbatim (monospace), convenient for an appendix or a robustness-check log.

8.1 Comparing multiple models: GMMStargazer

```

from pyxtabond2 import GMMStargazer

stargazer = GMMStargazer(
    [res1, res2, res3],
    model_names=["Diff (1-Step)", "Sys (1-Step)", "Sys (2-Step Robust)"],
)
stargazer.to_latex("comparison.tex")
stargazer.to_word("comparison.docx")

```

`GMMStargazer` builds a side-by-side comparison table (à la Stata's `esttab/estout` or R's `stargazer`): it takes the union of regressors, AR lags, and Diff-in-Sargan/Hansen tests across all supplied models (models missing a given row show -), and marks significance with $*p<0.1$, $**p<0.05$, $***p<0.01$. See Table 12 (Section 12) for a complete, real four-model instance of exactly this table.

9 IFE-GMM: the interactive-fixed-effects extension

9.1 Motivation

Standard Arellano-Bond/Blundell-Bond GMM assumes the idiosyncratic error is uncorrelated with the instruments. If the panel instead contains *unobserved common factors* with heterogeneous, unit-specific loadings – interactive fixed effects, in the sense of Bai [2009] – driving both the regressors and the dependent variable, that assumption fails and standard GMM is biased. IFE-GMM adds an iterative PCA-based defactoring step around the same GMM engine: extract candidate factors from current residuals, project the dependent variable, regressors, *and* instruments onto the space orthogonal to those factors, re-estimate GMM, and repeat until the coefficient vector stabilizes.

9.2 Parameters

r=0 (default) Plain GMM; no defactoring. Identical to omitting IFE-GMM entirely.
r=int Fixed number of factors. Each iteration: residuals $\rightarrow$ SVD $\rightarrow$ extract the top r factors F ($T \times r$, normalized so $F'F/T = I_r$, following Bai, 2009) $\rightarrow$ project with $M_F = I_T - FF'/T$ $\rightarrow$ re-run GMM on the projected data $\rightarrow$ check convergence.
r='auto' Estimates the number of factors first, from the plain-GMM residuals reshaped into an $N \times T$ matrix, via the Ahn and Horenstein [2013] eigenvalue-ratio criterion (with the Bai and Ng [2002] IC_{p2} criterion also computed and reported), mirroring Stata's `xtnumfac`, built using the same missing-cell handling described below.
r_max=5 Caps the factor search when **r='auto'**.
ife_max_iter=30, ife_tol=1e-5 Convergence-loop cap and tolerance on $\sum |\hat{\beta}_{\text{new}} - \hat{\beta}_{\text{old}}|$.
bias_correction=False Applies the Dhaene and Jochmans [2015] split-panel jackknife on top of the converged IFE-GMM fit: split the panel in half along the *time* dimension, re-run IFE-GMM on each half (warm-started from the full-panel solution), and combine as

$$\hat{\beta}_{\text{BC}} = 2\hat{\beta}_{\text{full}} - \frac{1}{2} \left(\hat{\beta}_{t \leq \bar{t}} + \hat{\beta}_{t > \bar{t}} \right),$$

correcting an $O(1/T)$ incidental-parameter-style asymptotic bias. Roughly doubles computation time (two half-panel refits).

Note. Any panel cell without a defined residual – a genuine gap in an unbalanced panel, or simply a variable's own first observation per unit (an undefined lag) – is filled automatically before each iteration's factor extraction, using the *previous* iteration's own rank- r reconstruction (a cross-sectional mean on the very first iteration, before one exists). `pyxtabond2` prints an informational, one-time console notice the first time this triggers per fit; it requires no user action and does not change the structure of the returned result. See the companion Methodological Note for the exact mechanism.

```

# Fixed factor count, with bias correction
model = PyXtabond2(df, id_col, time_col, dep_var, x_vars,
                   gmm_vars=['y'], iv_vars=['x1', 'x2'],
                   model_type='system', r=2, bias_correction=True)
result = model.fit()

# Automatic factor-count selection
model_auto = PyXtabond2(df, id_col, time_col, dep_var, x_vars,
                        gmm_vars=['y'], iv_vars=['x1', 'x2'],
                        model_type='system', r='auto', r_max=5)
result_auto = model_auto.fit()
model_auto.plot_factor_selection(mode='table') # or mode='graph'

```

`plot_factor_selection` only works after fitting with `r='auto'` (it reads criteria stored during that fit); it prints a table (returns a `DataFrame`) or a two-panel plot of the ER/IC_{p2} criteria across candidate factor counts (returns a `Figure`).

Note. During IFE-GMM fitting, progress is printed to the console (factor count selected, per-iteration convergence delta, jackknife status). This is informational only and does not affect the returned result.

9.3 Convergence and bias caveats

Two structural limitations apply to IFE-GMM and are documented here rather than silently assumed away (full theoretical treatment in the companion Methodological Note):

1. **Convergence is not guaranteed from an inconsistent starting point.** [Hong et al. \[2022\]](#) prove fast contraction to the true parameter for exactly this update formula, but only from a starting estimate that is already consistent under factors (they obtain one via nuclear-norm regularization). `pyxtabond2`'s loop instead starts from the plain (non-defactored) GMM fit, which is *not* guaranteed consistent in the presence of strong factors. Empirically the loop tends to behave well, but convergence should be read as a good empirical sign, not a correctness proof, especially with dominant factors.
2. **Without `bias_correction=True`, point estimates carry asymptotic bias – not variance.** [Hong et al. \[2022\]](#)'s Theorem 3.4 shows the iterative estimator's asymptotic variance already coincides with plain GMM at the true factors (no extra correction term is needed in `.se`), but its point estimate carries bias terms that vanish only under special cases (strictly exogenous instruments, no cross-sectional heteroskedasticity, no serial correlation/heteroskedasticity in the idiosyncratic errors) that need not hold in practice. **`bias_correction=True` is recommended for publication-grade inference** for exactly this reason.

10 Known limitations

- **Weights** (`pweight`/`aweight`/`fweight`) are **not implemented**. This is a deliberate scope decision, not an oversight: `xtabond2.mata` shows weights entering at least four distinct, non-trivial places (the W_1 accumulation, the error-variance scaling, the two-step "meat" matrix, and the AR tests), and only the first was verified with full rigor before the decision was made not to ship a partially-verified feature.
- **`cluster()` is single-variable only.** Stata's multi-way `cluster(var1 var2)` (Cameron-Gelbach-Miller alternating-sum clustering) is out of scope; most macro panels cluster on the panel id itself anyway.

- **cluster() combined with twostep/robust when instruments outnumber clusters:** the moment covariance matrix becomes exactly rank-deficient. NumPy’s Moore-Penrose pseudo-inverse and Stata Mata’s `invsym`-based generalized inverse are both *valid* generalized inverses of a singular matrix, but not numerically the *same* one – point estimates (not just standard errors) can then diverge from Stata. Confirmed to be specific to this singular regime (away from it, results match Stata exactly). Mitigation: keep the instrument count at or below the cluster count (e.g. via `collapse=True`) – the same mitigation Stata’s own on-screen warning for this situation implies.
- **GMMStyle/IVStyle’s Stata analogues `passthru`, `mz`, and `split` suboptions** are not implemented.
- IFE-GMM’s iterative scheme has no proven convergence guarantee from an inconsistent starting point (Section 9.3).
- IFE-GMM without `bias_correction=True` carries asymptotic bias in the point estimate (Section 9.3).
- **Unbalanced-panel Stata validation covers an interior gap and a late-starting individual** (6-decimal match on every coefficient and SE, Section 4). End-of-panel attrition is validated internally (no crash; finite, well-posed inference across all three AR-test weighting branches) but has not yet been independently re-run against Stata for that specific boundary case.
- **IFE-GMM’s missing-cell imputation** (Section 9) is reliable for light-to-moderate missingness but not independently stress-tested for panels with a large missing fraction, where the circularity of imputing from the model’s own evolving factor structure could compound the convergence caveat above.

11 Performance notes

- **`collapse=True` is the single biggest lever.** It reduces a GMM-style variable’s instrument column count from growing with T^2 to growing linearly in T . This matters even more for IFE-GMM, which internally re-runs the GMM engine at every iteration of its defactoring loop (and twice more per half-panel if `bias_correction=True`). Always consider `collapse=True` for simulation-heavy designs.
- The IFE-GMM loop defers all diagnostic computation (Sargan, Hansen, AR, Wald, Diff-Sargan) to the final iteration. This requires no configuration on the user’s part; it simply explains why per-iteration console output stays lightweight.

12 Worked examples

The package ships two runnable scripts in `examples/`: `examples.py` (standard GMM, four specifications, on `df_panel.csv`) and `ife_examples.py` (IFE-GMM, four specifications, on `df_ife_panel.csv`). Both are plain python `examples/<script>.py` runs and both end by exporting a `GMMStargazer` comparison table to L^AT_EX and Word.

12.1 Standard GMM: `examples.py`

`examples.py` walks a single macro panel through four increasingly rich specifications:

#	Specification
1	Difference GMM, one-step.
2	System GMM, one-step.
3	System GMM, two-step, robust, small (Windmeijer-corrected).
4	Model 3 + <code>collapse=True</code> .

All four results are then compared side by side with `GMMStargazer` and exported to both $\LaTeX$ and Word (Section 8). Table 12 reproduces that comparison exactly as `to_latex()` generates it for this script: a reproducible run on the bundled `df_panel.csv` dataset, not a hand-curated illustration.

Table 12: Four-specification comparison: actual output of `examples.py` via `GMMStargazer`

	Diff (1-Step)	Sys (1-Step)	Sys (2-Step Rob)	Sys (Collapsed)
L1.Growth	0.5614*** (25.770)	0.7978*** (75.331)	0.7777*** (34.408)	0.7054*** (22.677)
Capital	1.1459*** (11.160)	1.3634*** (12.530)	1.3087*** (9.779)	1.1893*** (2.726)
Labor	-0.0187 (-0.209)	-0.1529 (-1.639)	-0.2147 (-1.601)	0.2290 (0.698)
Wage	-0.0058 (-0.071)	-0.0452 (-0.531)	-0.0170 (-0.145)	0.3334 (0.875)
Investment	-0.1051 (-1.302)	-0.2205** (-2.441)	-0.1969 (-1.582)	-1.0465** (-2.090)
Ide	-0.0286 (-1.370)	-0.0367* (-1.661)	-0.0270 (-0.957)	-0.0270 (-0.642)
_cons		1.0923*** (2.620)	1.1020* (1.749)	1.2127 (0.726)
Observations	1600	1800	1800	1800
Number of groups	200	200	200	200
Instruments	73	90	90	20
AR(1) p-value	0.000	0.000	0.000	0.003
AR(2) p-value	0.198	0.056	0.048	0.055
Sargan p-value	0.251	0.000	0.000	0.734
Hansen p-value	-	-	0.001	0.704
Diff p-val: iv(Ide)	0.148	0.761	0.689	0.426
Diff p-val: for levels	-	0.000	0.000	0.065

Note: * $p < 0.1$; ** $p < 0.05$; *** $p < 0.01$. The z (or t) statistics are reported in parentheses.

Note. Each mechanism this table exercises – one-step vs. two-step weighting, the Windmeijer correction, and instrument collapsing – is validated individually, against a real Stata run or a fixed regression fixture, in the companion Methodological Note’s Stata-fidelity and validation sections. This four-model comparison is itself a reproducible worked example rather than a line-by-line Stata replication. See Section 9 for IFE-GMM (an original extension with no Stata equivalent, exercised separately by the companion `ife_exemples.py` script below) and Section 4 for Forward Orthogonal Deviations.

12.2 IFE-GMM: `ife_exemples.py`

`examples/ife_exemples.py` is the companion script for the interactive-fixed-effects extension (Section 9): it fits System GMM with IFE on the bundled `df_ife_panel.csv` dataset (built with a genuine multifactor error structure) across four specifications:

#	Specification
1	System GMM with IFE, one-step, fixed <code>r=2</code> .
2	System GMM with IFE, two-step, robust, small, fixed <code>r=2</code> (Windmeijer-corrected).
3	System GMM with IFE, two-step, robust, small, <code>r='auto'</code> (<code>r_max=5</code>) – automatic factor-count selection via <code>plot_factor_selection</code> instead of a user-fixed <code>r</code> .
4	Model 2 + <code>collapse=True</code> + <code>bias_correction=True</code> (split-panel jackknife).

Table 14 reproduces the actual `GMMStargazer` output of this script on the bundled dataset.

Table 14: IFE-GMM comparison: actual output of `ife_exemples.py` via `GMMStargazer`

	1-Step	2-Step Robust	Auto r	Bias Corrected
L1.Growth	0.5018*** (63.305)	0.5086*** (45.774)	0.5086*** (45.774)	0.5052*** (45.925)
Capital	1.2744*** (52.492)	1.3044*** (32.996)	1.3044*** (32.996)	1.3279*** (19.132)
_cons	-0.0093 (-0.557)	-0.0083 (-0.552)	-0.0083 (-0.552)	-0.0164 (-1.415)
Observations	3800	3800	3800	3800
Number of groups	200	200	200	200
Instruments	379	379	379	39
AR(1) p-value	0.000	0.000	0.000	0.000
AR(2) p-value	0.858	0.847	0.847	0.980
Sargan p-value	0.000	0.000	0.000	0.056
Hansen p-value	-	1.000	1.000	0.037
Diff p-val: GMM instruments for levels	0.000	1.000	1.000	0.640

Note: * $p < 0.1$; ** $p < 0.05$; *** $p < 0.01$. The z (or t) statistics are reported in parentheses.

Note. Column 3 (`r='auto'`) reports the *same* numbers as Column 2 (fixed `r=2`): on this dataset, both the ER [Ahn and Horenstein, 2013] and IC_{p2} [Bai and Ng, 2002] criteria independently select $r = 2$ (Figure 1), matching the value used elsewhere in this table. Automatic selection therefore reproduces the fixed-`r` fit rather than diverging from it – the expected outcome on a dataset built with a genuine two-factor structure.

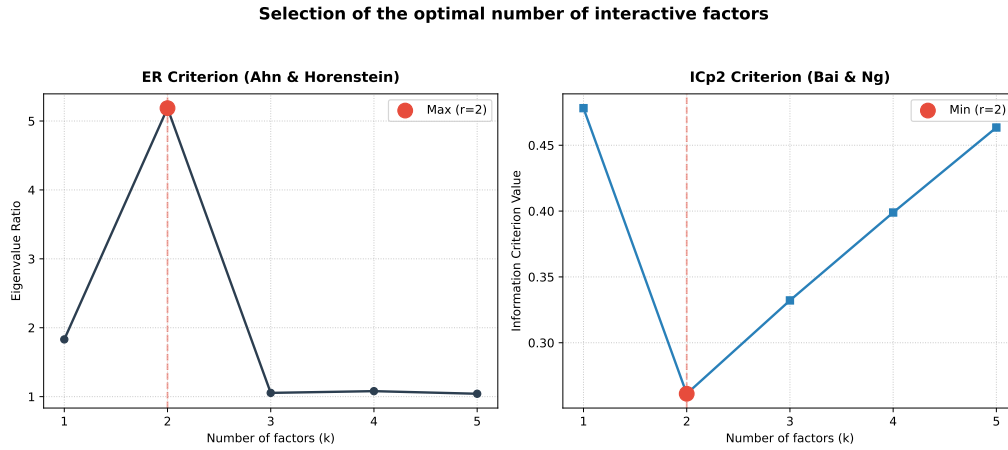


Figure 1: ER (Ahn-Horenstein) and IC_{p2} (Bai-Ng) factor-count criteria for Model 3, as returned by `model3.plot_factor_selection(mode='graph')` on the bundled `df_ife_panel.csv` dataset. Both criteria peak/trough at $r = 2$.

Note. Models 1–3 (uncollapsed, 379 instruments) converge within the default `ife_max_iter=30`. Model 4 (`collapse=True`) hits the 30-iteration cap on this dataset – the script prints **Warning: convergence not achieved after 30 iterations** and still returns the last iterate, per the convergence caveat of Section 9.3. This behavior is a property of the bundled example itself, not a curated success case. Raise `ife_max_iter` if your own data needs a tighter convergence guarantee.

12.3 Bundled datasets

```
from pyxtabond2 import list_datasets
list_datasets()
```

File	Description
<code>df_panel.csv</code>	Macro panel (Country/Year; Growth, Capital, Labor, Wage, Investment, Ide) used throughout the Quick Start and <code>examples.py</code> .
<code>df_ife_panel.csv</code>	Panel with a genuine multifactor error structure, used by <code>ife_examples.py</code> and for exercising <code>r/r='auto'</code> generally.

References

- Seung C. Ahn and Alex R. Horenstein. Eigenvalue ratio test for the number of factors. *Econometrica*, 81(3):1203–1227, 2013.
- Manuel Arellano and Stephen Bond. Some tests of specification for panel data: Monte Carlo evidence and an application to employment equations. *Review of Economic Studies*, 58(2): 277–297, 1991.
- Manuel Arellano and Olympia Bover. Another look at the instrumental variable estimation of error-components models. *Journal of Econometrics*, 68(1):29–51, 1995.
- Jushan Bai. Panel data models with interactive fixed effects. *Econometrica*, 77(4):1229–1279, 2009.
- Jushan Bai and Serena Ng. Determining the number of factors in approximate factor models. *Econometrica*, 70(1):191–221, 2002.
- Richard Blundell and Stephen Bond. Initial conditions and moment restrictions in dynamic panel data models. *Journal of Econometrics*, 87(1):115–143, 1998.
- Geert Dhaene and Koen Jochmans. Split-panel jackknife estimation of fixed-effect models. *Review of Economic Studies*, 82(3):991–1030, 2015.
- Jan Ditzen. Estimating dynamic common-correlated effects in Stata. *Stata Journal*, 18(3): 585–617, 2018.
- Shengjie Hong, Liangjun Su, and Tao Jiang. Profile GMM estimation of panel data models with interactive fixed effects. Working paper, July 25, 2022, 2022.
- Sebastian Kripfganz and Vasilis Sarafidis. Instrumental-variable estimation of large-T panel-data models with common factors. *Stata Journal*, 21(3):659–686, 2021.
- Milda Norkutė, Vasilis Sarafidis, Takashi Yamagata, and Guowei Cui. Instrumental variable estimation of dynamic linear panel data models with defactored regressors and a multifactor error structure. *Journal of Econometrics*, 220(2):416–446, 2021.
- M. Hashem Pesaran. Estimation and inference in large heterogeneous panels with a multifactor error structure. *Econometrica*, 74(4):967–1012, 2006.
- David Roodman. How to do xtabond2: An introduction to difference and system GMM in Stata. *Stata Journal*, 9(1):86–136, 2009a.
- David Roodman. A note on the theme of too many instruments. *Oxford Bulletin of Economics and Statistics*, 71(1):135–158, 2009b.
- Frank Windmeijer. A finite sample correction for the variance of linear efficient two-step GMM estimators. *Journal of Econometrics*, 126(1):25–51, 2005.